

FULL PROJECT DOCUMENTATION

BrainBarter

An AI-native peer-to-peer learning marketplace where students monetize knowledge and an AI tutor answers questions grounded in the actual uploaded material.

RAG Pipeline

Vector Search

Agentic AI

React · Node.js · Express

PostgreSQL · pgvector

Google Gemini API

Razorpay HMAC

Row-Level Security

SendGrid

Supabase

Vercel · Render

Live App

[brain-barter-](#)
[zeta.vercel.app](#)

Type

Full-Stack Web
Application

Stack

React · Express · Supabase ·
Gemini

Year

2025–
2026

BrainBarter is a student-powered knowledge marketplace built on a token economy: students **earn tokens by uploading** high-quality study material (videos and notes), and **spend tokens to unlock** content created by their peers. Creators can withdraw accumulated earnings directly to a UPI or bank account. Layered on top of this economy is a **Retrieval-Augmented Generation (RAG) AI tutor** — powered by Google Gemini — that reads the actual uploaded PDFs and answers student doubts with citations to the exact passages it used, rather than generating answers from general knowledge.

For Learners

Browse and discover content by subject, topic, type, and difficulty. Unlock material using tokens, rate content, bookmark topics, and track progress. Start with 50 free tokens on signup.

For Creators

Upload videos or PDF notes with subject and topic tags. Earn tokens every time someone unlocks your content. Manage uploads through a creator dashboard. Withdraw earnings to real money.

AI Study Assistant

RAG-powered doubt solver, summarizer, note generator, and practice question engine — all grounded in the student's own uploaded material with inline citations to source passages.

Secure by Design

HMAC-SHA256 payment verification, atomic SECURITY DEFINER DB functions, Row-Level Security on every table, and cryptographic email OTPs — clients cannot manipulate token balances.

FRONTEND

React 19 + Vite

Component-based UI with fast HMR. React Router for SPA navigation, Zustand for lightweight global state (auth + theme), React Hot Toast for notifications.

STYLING

Tailwind CSS

Utility-first CSS with a custom design system — brand gradients, dark mode tokens, glass-morphism cards, and responsive layout utilities.

BACKEND

Node.js + Express 5

RESTful API server. Handles AI orchestration, payment verification, file ingestion, and all token operations via SECURITY DEFINER RPCs.

DATABASE & AUTH

Supabase / PostgreSQL

Managed Postgres with Row-Level Security on every table. Supabase Auth for JWT sessions. Supabase Storage for file uploads (PDFs, videos).

VECTOR SEARCH

pgvector

IVFFlat index (100 lists) on 768-dim Gemini embeddings stored in the `content_chunks` table. Cosine-similarity search via `match_content_chunks()` RPC.

AI / ML

Google Gemini API

`text-embedding-004` for 768-dim vector embeddings. `gemini-1.5-flash` for all generative tasks — doubts, summaries, mock tests, study plans.

PAYMENTS

Razorpay

Order creation + HMAC-SHA256 webhook verification. Idempotent crediting via unique `payment_id` prevents any double-credit on retried callbacks.

EMAIL

SendGrid

Transactional email for 6-digit OTP delivery during signup and password reset. Codes generated with `crypto.randomInt()`, stored in Supabase with 10-min expiry.

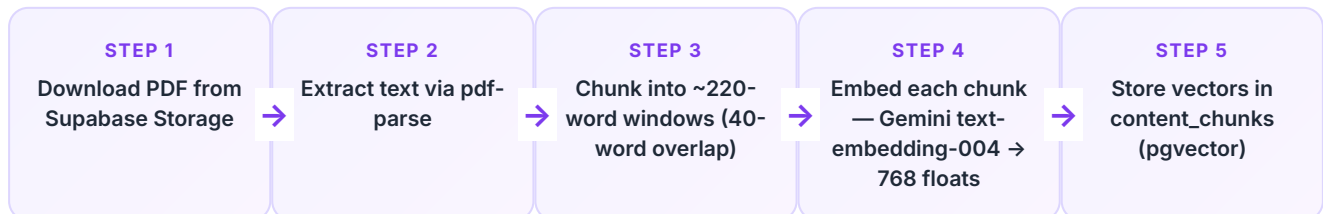
DEPLOYMENT

Vercel + Render

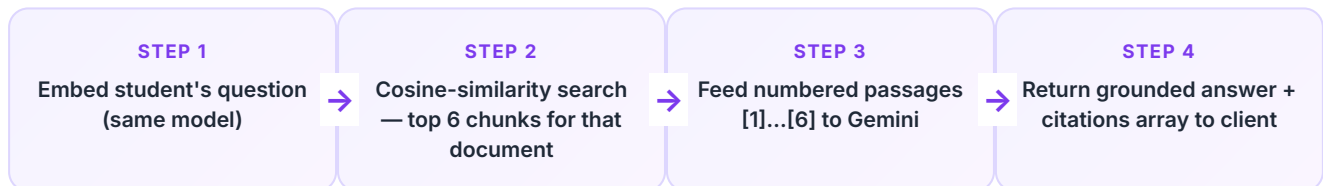
Frontend on Vercel (auto-deploy from GitHub, CDN-distributed). Backend on Render (Node.js service, auto-deploy on push). Separate CORS-controlled origins.

Standard AI chatbots answer from training data — BrainBarter's AI answers exclusively from the student's uploaded PDF, making every answer verifiable and grounded in the actual syllabus material.

Ingestion Phase (run once after upload)



Retrieval Phase (on every doubt question)



```

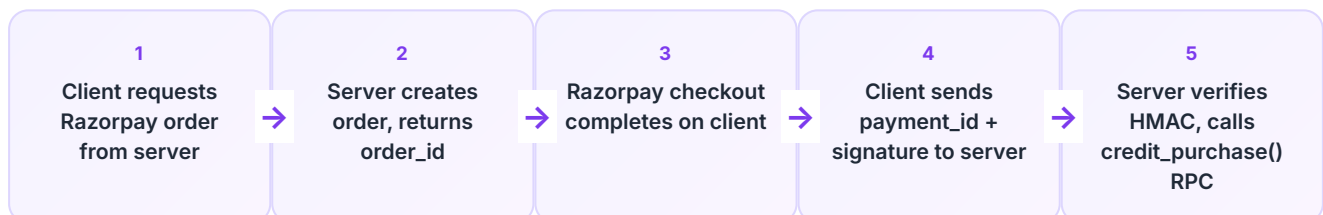
// server/utils/embeddings.js - chunking with overlap
function chunkText(text, size = 220, overlap = 40) {
  const words = text.replace(/\s+/g, ' ').trim().split(' ')
  const chunks = []
  for (let i = 0; i < words.length; i += size - overlap) {
    chunks.push(words.slice(i, i + size).join(' ').trim())
    if (i + size ≥ words.length) break
  }
  return chunks
}

// server/controllers/aiController.js - semantic retrieval for doubts
const queryEmbedding = await embedText(userMessage)
const { data: matches } = await supabase.rpc('match_content_chunks', {
  p_content_id: contentId,
  query_embedding: queryEmbedding,
  match_count: 6,
})
contentText = matches.map((m, i) => `[${i+1}] ${m.chunk_text}`).join('\n\n')
citations = matches.map((m, i) => ({ ref: i+1, snippet: m.chunk_text.slice(0, 180), similarity: m.sim
  
```

Security & Engineering Highlights

Implementation	What it does & why it matters
HMAC-SHA256 Payment Verification	Every Razorpay callback is verified server-side by recomputing the HMAC signature. A client that forges a payment ID cannot credit tokens.
Idempotent Token Crediting	<code>credit_purchase()</code> is a <code>SECURITY DEFINER</code> function that checks <code>UNIQUE(payment_id)</code> before crediting — a retried or duplicated webhook callback can never double-credit tokens.
Row-Level Security (RLS)	Enabled on every table. The <code>profiles</code> policy explicitly blocks clients from updating <code>token_balance</code> or <code>role</code> — even with a valid JWT. All balance changes go through audited RPCs.
Atomic Withdrawal RPC	<code>request_withdrawal()</code> deducts the token balance and inserts the withdrawal record atomically in one DB transaction, preventing double-spend from concurrent requests.
Self-Rating Prevention	<code>rate_content()</code> <code>SECURITY DEFINER</code> function checks <code>creator_id ≠ rater_id</code> at the DB level — a creator cannot earn the +2 token rating reward on their own content.
Cryptographic Email OTPs	6-digit codes generated via <code>crypto.randomInt()</code> (CSPRNG), stored in Supabase (not in-memory), expire in 10 minutes, and are deleted on successful use — resilient to cold-start restarts.
Creator-Only Ingestion Guard	The <code>/api/ai/ingest</code> route verifies <code>content.creator_id ≡ req.user.id</code> before running the embedding pipeline — preventing anyone from triggering costly GPU work on another user's content.
First-Rating Idempotency	The +2 token reward for rating is gated by a DB check for an existing rating row — submitting the same rating twice never double-credits the rater.

Payment Flow (end-to-end)





RAG Doubt Solver

Ask any question about an uploaded PDF. The AI retrieves the most relevant chunks via cosine similarity and answers exclusively from those passages, citing [1], [2]... inline. Displayed with a "Grounded in this material" citation panel.

Summarize & Simplify

Condenses any uploaded content into 5–7 bullet points, or re-explains it in plain language. Runs against the full indexed text of the document (not just the title/description).

Generate Notes & Questions

Produces structured revision notes with headings and definitions, plus 10 exam questions (short + long answer) derived directly from the uploaded material.

Exam Mode (4 modes)

Expected Questions — probable exam questions using PYQ context. **Mock Test** — 10-question MCQ with answers. **Revision Sheet** — one-page concept summary. **Final Prep** — must-revise checklist.

AI Study Planner (Agentic)

Given a subject, optional topic, and target days (3–30), Gemini generates a day-by-day revision plan with daily goals, subtopics, activities (reading, practice, flashcards), and a daily self-check question. Progressive difficulty across the plan.

Previous Year Question Context

Exam Mode pulls stored PYQ text dumps from the database and feeds them as additional context to Gemini — so expected questions are calibrated to actual exam patterns, not generic guesses.

TOKEN RULES

Buy rate	₹1 = 10 tokens
Sell / withdraw rate	10 tokens = ₹0.80
Platform fee	20% on withdrawals
Minimum purchase	₹10 (100 tokens)
Minimum withdrawal	₹50 (625 tokens)
Signup bonus	50 free tokens
First-rating reward	+2 tokens

- ▶ **Earn:** Upload content → receive tokens every time a learner unlocks it
- ▶ **Spend:** Unlock any content with tokens via atomic server-side transaction
- ▶ **Withdraw:** Submit UPI or bank details → admin approves → real INR transferred
- ▶ **Buy:** Top up via Razorpay — ₹10 minimum, verified server-side
- ▶ **Reject refund:** If admin rejects a withdrawal, tokens are automatically refunded via `refund_tokens()` SECURITY DEFINER function
- ▶ **Full ledger:** Every earn, spend, buy, and withdrawal is recorded with timestamp and type in a transaction history table

Withdrawals

Review pending withdrawal requests. Approve to disburse, reject to auto-refund tokens to creator via DB function.

Feedback

Triage user-submitted feedback. Mark items as Reviewed or Done. Supports category filtering (Bug, Feature, UX, etc.).

Reports

Review content reports submitted by users. Take action: warn creator, unpublish content, or dismiss the report.

Verify Creators

Grant verified-creator badges to qualified uploaders — displayed as a badge on their profile and content.

Users

View all registered users, manage roles, and monitor account activity across the platform.

Access Control

Admin tab is protected by both client-side `ProtectedRoute` `adminOnly` and server-side role check from JWT claims via Supabase RLS.

Authentication & Accounts

Email-verified signup — 6-digit OTP generated via `crypto.randomInt()`, sent through SendGrid, stored in Supabase with 10-minute TTL. Codes are deleted on successful verification. Resistant to cold-start restarts (no in-memory state).

Password reset via email — same OTP flow. User receives a code, enters it to confirm identity, then sets a new password via the Supabase Auth API.

Role-based access — roles are `user` or `admin`. The admin role is set in the database and propagated through JWT claims. RLS policies and API guards check this role at every sensitive operation.

50 starter tokens on signup — a PostgreSQL trigger (`handle_new_user()`) fires on `auth.users` insert and credits 50 tokens to the new profile automatically.

Resume-Ready Summary

BRAINBARTER — PEER-TO-PEER LEARNING MARKETPLACE

- Built a **peer-to-peer learning marketplace** where students earn tokens by uploading study material and spend them to unlock peer content, with real-money withdrawals via UPI/bank — secured by **HMAC-SHA256 Razorpay verification**, **atomic PostgreSQL transactions**, and **Row-Level Security** preventing any client-side token manipulation
- Implemented a **RAG AI pipeline** using Google Gemini embeddings (768-dim) + pgvector cosine similarity so the AI tutor answers student doubts grounded in the actual uploaded PDF, returning **inline citations** to the exact passages used — not general training knowledge
- Engineered an **agentic AI Study Planner** that generates personalized day-by-day revision schedules (3–30 days) with progressive goals and daily self-checks, alongside **Exam Mode** (mock tests, revision sheets, expected questions) — full stack: React · Express · Supabase · Gemini · Razorpay

Live: brain-barter-zeta.vercel.app · **Tech:** React · Vite · Tailwind · Node.js · Express · PostgreSQL · pgvector · Supabase · Google Gemini API · Razorpay · SendGrid · Vercel · Render

System Architecture

